Audit v3 — Complete Redesign Spec

SubSense · Design Spec · 2026-02-21（修订版） **Status:** Ready for implementation
Replaces: audit-v2-spec.md.pdf

修订记录 (2026-02-21):

1. §1.2 Need Fit — 字数代理指标替换为 Claude 分类的结构化清晰度信号（scenario_specific / problem_aware / category_only / none）
2. §1.4 Budget Fit — 比例调整现在按用户 price_sensitivity persona 缩放（固定 ±3 → 自适应 -5 到 +3）
3. §1.4 Cost-per-use — 固定 5阈值替换为按类别基准线（*entertainment*0.50, productivity $2 等）
4. §1.6 Independent Risk — "Not sure" 分数从 8 降到 5（低于主动选择的风险选项）
5. §2.2 History check — 系统先查现有订阅记录再提问；Claude 根据已知上下文调整问题措辞
6. §1.9/4.2 评分区间 — Subscribe Now 阈值从 70 提高到 75；所有区间上移；分数曲线计划在 Phase 2 实施
7. §4.3 Why 板块 — 单段落替换为结构化 Pro/Con 格式，对齐雷达图维度
8. §4.5 Next Steps — 新增完整导航 UX 规范（action 类型、流程、返回行为、状态保持）

待改动清单

优先级	改动	类别
P0	Risk 维度去重复，5 个新独立选项	评分逻辑
P0	Need Fit 改为清晰度打分（去 strong/weak 偏见）	评分逻辑
P0	Budget Fit 感受为主 + cost-per-use 微调	评分逻辑
P0	Need Fit × Usage 交叉验证修正	评分逻辑
P0	Not Sure 分slot加权惩罚	评分逻辑
P1	新增 Stage 2 `history_check`	对话流程
P1	报告三层结构 + 渐进式展示	报告输出
P1	Why 段落 Claude 生成（8种场景模板 + prompt 约束）	报告输出
P1	Next Steps 动态生成（4种推荐路径 × 用户数据填充）	报告输出
P1	Watch Items 去重逻辑（vs 对话 Aha）	报告输出

优先级	改动	类别
P1	雷达图组件（折叠展示）	报告输出
P1	分数标尺（0-40-55-70-100 可视化）	报告输出
P1	对话中 Aha 独立卡片视觉 + 扩大触发条件	对话体验
P2	Echo Back — Bot 复述用户回答	对话体验
P2	中途 Progress Feedback 总结	对话体验
P2	条件性幽默（安全场景下）	对话体验
P2	Stage 4 Slider 输入形态	对话体验
P2	Overlap 问题引用用户现有订阅名称	对话体验
P2	Motivation 追问逻辑	对话体验
P2	Budget Feel 带上下文数字	对话体验
P2	Claude prompt banned phrases 列表	文案质量
P2	文案硬限制（Verdict ≤15词，Why ≤60词）	文案质量
P2	v2 调节器（Persona Alignment / Lifecycle / Portfolio Pattern）	评分逻辑
P3	对话开场白路径分叉（基于 Lifecycle）	对话体验
P3	Check-in 数据接入 Audit	数据集成

结构

章节	内容
Part 1	评分逻辑 — 5 维度新权重、Need Fit 清晰度打分、Budget 感受为主、Risk 去重复、交叉验证、加权 uncertainty、cost-per-use
Part 2	对话流程 — 7 阶段定义、history_check 新增、Slider 输入、motivation 追问、mid-conversation 总结、Echo Back 和条件幽默
Part 3	Aha 系统 — 对话 Aha 卡片视觉、10 种触发条件、triggeredInsights 追踪、三层去重规则
Part 4	报告输出 — 三层渐进式展示、Verdict 模板、Why 8 种场景示例、雷达图设计、Next Steps 4 种路径、Watch Items 11 种条件模板、文案硬限制和 banned phrases
Part 5	Claude Prompt — 对话 system prompt 完整版、报告 Why 和 Verdict 的 prompt 规则

章节	内容
Part 6	数据流 — Aha 信号生命周期图、完整 audit 数据流图、CTA 行为反哺
Part 7	实施计划 — 文件映射、P0/P1/P2/P3 优先级排序
Appendix A	完整的 TypeScript 类型定义（可直接复制进代码）
Appendix B	遗留问题清单

Background: 为什么需要 v3

v2 提出了正确的问题——Audit 太冷漠、太模板化——并设计了三个 context adjustor（Persona Alignment、Lifecycle Signal、Portfolio Pattern）。但当前代码只部分实现了 v2，并且有几个结构性问题：

1. **Risk 维度和其他维度重复** — Risk 的 4 个选项中有 3 个重复了 Usage、Budget、Overlap 已经采集的信号，等于同一个信号被罚了两次分
2. **Need Fit 有类别偏见** — entertainment 被惩罚，work 被奖励，不管用户描述得多清晰
3. **Budget 评分以比例为主** — 用户自己说的感受反而是次要的微调，权重倒挂
4. **Aha Moment 功能不足** — 只有 4 种触发条件，每次 Audit 最多 1 次，视觉上和普通聊天气泡一样
5. **报告输出是死的** — 4 条固定推荐文案，factor 描述空洞，没有个性化
6. **对话像做问卷** — 6 轮点 chip，没有节奏变化，没有"被听到"的感觉

v3 修复以上所有问题，同时整合 v2 的 adjustor 架构。

Part 1: Scoring Logic（评分逻辑）

1.1 Dimension Weights（维度权重，更新后）

Dimension	Max Score	相比 v1 的变化	理由
Need Fit	25	评分标准变了	不再按类别打分，改为按"动机清晰度"打分

Dimension	Max Score	相比 v1 的变化	理由
Expected Usage	25	不变	使用频率仍然是最强的预测信号
Budget Fit	20	从 25 降到 20	因为用户的主观感受 `budget_feel` 本身就是最直接的信号，不需要额外的比例计算占太多权重
Overlap	15	不变	保留 v2 的 Portfolio Pattern adjustor
Independent Risk	15	从 10 升到 15	因为现在 Risk 的选项是真正独立的信号了（不再和其他维度重复），值得更多权重

满分：100 分

1.2 Need Fit (0–25): 基于清晰度的评分

改了什么

旧逻辑按动机*类别*打分（work=22, entertainment=14）。新逻辑按动机*清晰度*打分——用户能不能说清楚为什么想要这个订阅。

核心理念：**一个能具体描述"我每天晚上看两小时"的人，比一个只说"for work"的人更可能真正用下去。** 不再惩罚 entertainment，而是惩罚"说不清楚为什么想要"。

为什么不按字数打分?

之前的 v3 草案用字数（>10 字 = 高分，≤10 字 = 中分）来衡量清晰度。这有几个严重问题：

1. **用户看不到这个规则** — 字数对用户是隐形的，评分感觉随机且不可复现
2. **语言量 ≠ 意图质量** — "我每天晚上看"（6 个字）比"我觉得有了可能偶尔会用来做各种娱乐相关的事"（18 个字）更清晰
3. **语言和文化偏差** — 表达简洁的用户或非母语用户可能写得短但同样清楚

新方法：结构化清晰度信号

不计字数，改为基于 **结构化信号** 来判断需求的真实性。这些信号由 Claude 在 slot 提取过程中识别，基于购买意图预测的行为心理学研究（具体性、时间框架、场景具象性 — Locke & Latham, 2002 目标设定理论）。

Base score 规则

Claude 检测到的信号	Base score	为什么这个信号重要
场景具体型：用户描述了具体的使用情境、时间或工作流（如"我每周末用它剪 YouTube 视频"）	22	具体性是行为跟进的 #1 预测因子。能说出*什么时候*、*怎么用*的人，已经在心理上把服务融入了自己的生活。
问题感知型：用户指出了一个缺口或痛点，但没有具体场景（如"我需要一个剪辑工具"或"现在的太慢了"）	18	问题意识说明有真实摩擦。用户知道哪里不对但还没完全想清楚方案 — 中等信心。
类别标识型：用户只选了 motivation chip 没有补充，或给了泛泛的描述（如"for entertainment"、"可能有用"）	14	用户知道这是什么领域但没有和自己的生活联系起来。这是"逛街"信号 — 有兴趣但投入度不明。
无清晰度：选了 Not sure 或没回答	10	如果你说不出为什么想要某个东西，持续使用的可能性很低。

Claude 怎么检测这些信号

在 system prompt 中，Claude 被要求在提取 motivation 时同时分类 `motivation_clarity`：

```
When extracting motivation, also classify motivation_clarity:
- "scenario_specific": User mentioned a specific time, place, frequency,
  workflow, or personal context for using this service
  用户提到了具体的时间、地点、频率、工作流或个人使用场景
- "problem_aware": User described a problem, gap, or limitation they're
  trying to solve, but without specific usage scenario
  用户描述了要解决的问题或不足，但没有具体使用场景
- "category_only": User only identified a category/domain or gave a
  vague generic statement
  用户只标识了类别/领域，或给了模糊泛泛的描述
- "none": User said "not sure" or didn't answer

Classification is based on semantic content, NOT word count.
分类基于语义内容，不基于字数。
Examples:
  "I watch every night" → scenario_specific (5 words, but specific)
  "it seems interesting for various purposes" → category_only (6 words, but vague)
  "my current editor crashes when I export 4K" → problem_aware (specific problem)
```

为什么这样更好

1. **透明** — 评分标准和用户直觉一致（具体 vs 模糊），不是一个隐形指标
2. **跨用户类型稳健** — 对话多的和话少的、母语和非母语用户同样有效
3. **有研究支撑** — 具体性作为行为跟进预测因子在目标设定理论中有充分验证
4. **Claude 原生** — 利用 Claude 的语义理解能力，而不是粗暴的字数计数

Persona Alignment adjustor（来自 v2 spec）

算完 base score 后，乘以 Persona Alignment 系数（0.7~1.3）：

```
adjustedNeedFit = min(baseNeedFit × personaAlignmentCoeff, 25)
```

情况	系数	示例
强匹配：订阅类别和 Setup 里选的兴趣标签高度重合	1.2–1.3	音乐爱好者 audit Spotify
中性：无明显关联	1.0	生产力用户 audit Netflix
弱匹配：订阅类别和已知习惯明显不符	0.8–0.9	轻度消费者 audit 重度专业工具
警告：同类别已有 2+ 个订阅且使用率低	0.7	已有 3 个 entertainment sub，check-in 满意度都很低

数据来源： Setup 阶段的兴趣标签（v2 spec Part 2 设计的 "What areas do you tend to spend on most?"）+ subscriber type。

1.3 Expected Usage (0–25): 频率评分

Base score（和 v1 一样）

Frequency bucket	Score
0 (Rarely/never)	0
1 (About once/week)	10
2.5 (2–3 times/week)	18
4.5 (4–5 times/week)	23
7 (Daily)	25

Frequency bucket	Score
not_sure	10

Lifecycle Signal adjustor（来自 v2 spec）

核心理念：**历史行为比自我预测更可信。** 用户说"我会每天用"，但如果他上次因为"用太少"取消了同一个服务，这个预测的可信度要打折。

```
adjustedUsage = min(baseUsage × lifecycleCoeff, 25)
```

Lifecycle 状态	系数	数据来源
首次接触（无历史记录）	1.0	默认，依赖自我预测
订过，因"用太少"取消	0.6	历史行为覆盖预测
订过，因"经济压力"取消	1.1	意愿真实，时机问题
订过，找到了更好替代品	1.0	触发追问而非调整系数
正在用，想升级	跳过频率问题	直接给 23 分

数据来源： 新增的 Stage 2 `history_check` + 数据库中的订阅历史。

1.4 Budget Fit (0–20): 感受优先的评分

改了什么

旧逻辑：比例（price/budget）决定基础分 → 感受只是 ±5/-10 的微调。 新逻辑：**用户感受决定基础分** → 比例只提供微调。

理由：用户自己说"comfortable"，这就是最直接的信号——不管比例算出来多少。Setup 时填的月预算可能是随手填的、可能是专项预算、可能已经过时了。

`budget_feel` 决定 base score

budget_feel	Base score	理由
comfortable	18	用户的直接判断——最可信

budget_feel	Base score	理由
tight	10	能承受但有压力
no_way	3	明确拒绝
not_sure	10	中性默认

比例调整：按 Persona 自适应（替代固定 ±3）

为什么不用固定 ±3?

固定 ±3 把所有用户当成一样的人，但价格敏感度在不同 persona 之间差异巨大：

- **frugal_optimizer** 说 "comfortable"（ratio=18%）→ 大概率低估了，这类用户后面容易后悔
- **premium_explorer** 说 "comfortable"（ratio=18%）→ 真心觉得没问题，高消费已经内化为习惯
- **cautious_saver** 说 "tight"（ratio=5%）→ 确实紧张，即使绝对值不大

一刀切的调整无法捕捉这些差异。

新方案：按价格敏感度缩放调整幅度

比例调整的幅度随用户的 `price_sensitivity` 分数缩放（来自 Setup 中的 `budget_tightness` + `price_sensitivity`，范围 0–1，1=最敏感）。

```
ratio = price_monthly / monthly_budget
priceSensitivity = user.setupPreferences.price_sensitivity  // 0.0–1.0

// 基础调整（和之前一样）
if ratio < 0.05:
  ratioAdjust = +3
elif 0.05 ≤ ratio ≤ 0.15:
  ratioAdjust = 0
elif ratio > 0.15:
  ratioAdjust = −3

// 按敏感度缩放
// 高敏感度 (0.8–1.0)：惩罚放大 ×1.5，奖励缩小 ×0.5
// 低敏感度 (0.0–0.3)：惩罚缩小 ×0.5，奖励保持 ×1.0
// 中等敏感度 (0.3–0.8)：不缩放 (×1.0)

if ratioAdjust < 0:  // 惩罚
  scaleFactor = 0.5 + (priceSensitivity × 1.0)  // 范围: 0.5–1.5
  ratioAdjust = round(ratioAdjust × scaleFactor)
elif ratioAdjust > 0:  // 奖励
  scaleFactor = 1.0 − (priceSensitivity × 0.5)  // 范围: 0.5–1.0
  ratioAdjust = round(ratioAdjust × scaleFactor)
```

```
// 限制在 [-5, +3] 范围内
ratioAdjust = clamp(ratioAdjust, -5, +3)
```

各 persona 类型的效果

Persona	price_sensitivity	ratio>0.15 的惩罚	ratio<0.05 的奖励	理由
frugal_optimizer	0.9	−4 到 −5	+2	价格敏感用户被贵价比例打更狠 — 他们说的 "comfortable" 可能过于乐观
cautious_saver	0.8	−4	+2	类似 frugal 但稍温和
balanced_user	0.5	−3	+3	基线行为 — 就是原来的 ±3
relaxed_subscriber	0.3	−2	+3	对价格不太敏感；比例对他们的满意度影响更小
premium_explorer	0.1	−2	+3	高消费者不会被高比例吓到；便宜也不会特别让他们兴奋

兜底

如果 `price_sensitivity` 不可用（用户没做 Setup），默认 `0.5`（balanced），复现原来的 ±3 行为。确保向后兼容。

Cost-per-use 调整：按类别自适应（替代固定 $5 阈值）

为什么不用固定 $5 阈值?

不同产品类别的成本结构根本不同：

- **流媒体**（10–15/月，每天用）：$cost-per-use$ 天然 <1 — $5 的阈值永远不会触发
- **专业工具**（30–50/月，每周2−3次）：3–5/次是正常且可接受的
- **低频工具**（5–10/月，每月用一次）：即使5/次也合理，因为每次使用价值高（如报税软件、域名注册）

一个固定阈值无法覆盖这些差异。

新方案：按类别基准线比较

不用绝对美元阈值，改为和**类别特定基准线**比较。基准线代表该产品类别"健康使用模式"下的中位 cost-per-use，基于行业定价规范。

```
if price_monthly AND expected_frequency 都已知:
  monthlyUses = expected_frequency × 4.3
  costPerUse = price_monthly / max(monthlyUses, 1)

  // 类别基准线（健康使用模式下的中位 $/次）
  CATEGORY_BENCHMARKS = {
    entertainment:  0.50,   // 流媒体：每天用时每次几分钱
    productivity:   2.00,   // 工具：每次几美元是正常的
    health:         3.00,   // 健身 app：~$3/次和健身房竞争
    learning:       4.00,   // 学习平台：每次较高是预期中的
    social:         0.30,   // 通讯：接近零成本
    creative:       3.00,   // 设计工具：类似 productivity
    utilities:      5.00,   // 低频高价值（云存储、VPN）
    default:        2.00,   // 类别未知时的兜底
  }

  benchmark = CATEGORY_BENCHMARKS[category] or CATEGORY_BENCHMARKS.default

  // 基于和基准线的比值调整
  ratio_to_benchmark = costPerUse / benchmark

  if ratio_to_benchmark > 3.0:   -3   (3倍以上基准线 — 真正的低价值密度)
  elif ratio_to_benchmark > 1.5: -1   (略高于基准线 — 轻微关注)
  elif ratio_to_benchmark < 0.3: +2   (远低于基准线 — 优秀的价值密度)
  else:                           0   (在该类别正常范围内)
```

设计理由

1. **基准线有据可依** — 每个基准线映射到真实定价。Spotify 每天用 ≈ 0.33/次；*AdobeCC*每周3次 ≈3.60/次；两者用户都认为合理
2. **相对比较而非绝对比较** — "这个对它来说贵吗"而不是"这个贵吗"
3. **优雅退化** — 类别未知时回退到 $2.00 基准线，这是所有 SaaS 的合理中位数
4. **3 倍才惩罚** — 只在 3 倍基准线时才全额惩罚（而非 1.5 倍），因为一定程度的偏差是正常的

条件触发（优化）

避免在 cost-per-use 明显不相关时做不必要的计算：

```
// 以下情况跳过 cost-per-use 计算：
// 1. 用户说 budget_feel = "no_way"（预算已经是主要问题了）
// 2. expected_frequency = daily（cost-per-use 几乎一定很低）
```

```
// 3. price_monthly < 3（太便宜了，cost-per-use 无所谓）

skipCostPerUse = (budget_feel === 'no_way')
               || (expected_frequency >= 7)
               || (price_monthly < 3)
```

最终 Budget Fit

```
budget_fit = min(20, max(0, baseFeel + ratioAdjust + costPerUseAdjust))
```

1.5 Overlap (0–15): 带 Portfolio Pattern Adjustor

Base score（来自 `overlap_level`）

overlap_level	Score
none	15
some	10
significant	4
high	0
not_sure	7

Portfolio Pattern adjustor（来自 v2 spec）

触发条件：

```
sameCategory.count >= 2 AND sameCategory.avgUsageRate < 0.4
```

意思是：**同类别已经有 2 个以上的订阅，而且平均使用率低于 40%** → 说明用户有"在某个类别囤积订阅但不怎么用"的 pattern。

触发后：base Overlap 额外减 3 分（下限 0）。

同时触发报告里的反思问题（不只是扣分，还要让用户思考）。

无现有订阅的 bonus（保留自 v1）

如果用户目前零订阅：`overlap = max(overlap, 13)`

1.6 Independent Risk (0–15): 重新设计的选项

改了什么

旧 Risk 选项和其他维度重复：

- "Too expensive" → 和 Budget Fit 重复
- "Already have something similar" → 和 Overlap 重复
- "Won't use it enough" → 和 Expected Usage 重复

新选项只捕捉**其他维度完全没覆盖到的独立风险信号**。

新选项和评分

选项	value	Score	捕捉的信号
No real concerns	`none`	15	正面信号——用户没有犹豫
The free version might be enough	`free_enough`	10	付费 vs 免费的 ROI 质疑
I might forget to cancel	`forget_cancel`	7	取消摩擦（唯一保留的旧选项，因为确实独立）
I tend to lose interest quickly	`interest_decay`	6	兴趣衰减速度——比 low_usage 更有预测力
Not sure	`not_sure`	5	见下方理由
I'm being impulsive right now	`impulse`	4	冲动自知——用户主动承认，极其宝贵的信号

为什么 "Not sure" 得 5 分而不是 8 分

之前的草案给 `not_sure` 打 8 分（介于 `free_enough`=10 和 `forget_cancel`=7 之间）。这造成了一个悖论：**一个说不出任何风险的用户比一个主动说出 `forget_cancel` 或 `interest_decay` 的用户得分更高。**

这是错误的，原因：

1. **主动识别风险是正面信号** — 说"我容易失去兴趣"的用户展示了自我觉察力。他们做了评估自身行为模式的心理工作。这种自我觉察相对于无知应该被*奖励*，而不是被惩罚。

2. **"Not sure" 是模糊的，不是安全的** — "Not sure" 可能意味着：

   - "我没想过这个问题"（缺乏反思 → 有风险）
   - "我真的没有顾虑"（应该选 `none`）
   - "有风险但我说不出来"（未明确的风险 → 有风险） 3 种解读中有 2 种指向比主动选择*更低*的自我觉察。

3. **评分原则：已识别风险 > 未识别风险** — 在风险管理中，一个你命名了、能监控的已知风险，比一个你没考虑到的未知风险更安全。评分应反映这一点：`forget_cancel`(7) > `not_sure`(5)，因为"我知道该注意什么" > "我不知道我不知道什么"。

分数排序逻辑（从高到低）

```
none (15)           → "我考虑过了，没有顾虑" — 最好的信号
free_enough (10)    → 具体的、可解决的顾虑 — 不是 dealbreaker
forget_cancel (7)   → 已知风险，有明确的缓解手段（设提醒）
interest_decay (6)  → 有自我觉察的行为模式 — 诚实但令人担忧
not_sure (5)        → 未知风险状态 — 比命名了的风险更糟
impulse (4)         → 最强红旗 — 用户主动警告我们
```

Stage 位置

Risk 移到 **Stage 7**（最后一个问题）。问法：

> "Besides what we've already talked about, is there anything that makes you hesitate?"

"Besides" 这个词框定了范围——用户不会重复前面说过的内容。

1.7 Cross-Validation: Need Fit × Usage（交叉验证）

问题

Need Fit 和 Usage 正相关（动机清晰 → 预测使用频率也高）。两个都高或都低时互相放大，导致 55-69 分区间（Start Trial）很窄——大部分人不是 70+ 就是 <50。

修正

两个维度都算完后，做一次交叉检查：

```
if (need_fit > 18 AND usage < 12):
  // "想要但可能不会真的用"—动机可能是想象出来的
  need_fit -= 3

if (need_fit < 14 AND usage > 18):
  // "说不清为什么要但确实会常用"—行为比言语更可信
  usage += 2
```

1.8 Uncertainty Penalty: 按 Slot 加权（不再固定 2 分）

改了什么

旧逻辑：每次选 "not sure" 固定扣 2 分。 新逻辑：**哪个问题说了 not sure，惩罚不同。**

Slot	Penalty	理由
expected_frequency	−3	连自己会不会用都不知道 = 最高风险
budget_feel	−2	可能只是没算过，不代表付不起
primary_motivation	−2	说不清为什么想要 = 中等风险
overlap_level	−1	系统可以帮查，不全怪用户
top_risk	−1	"Not sure" ≈ "no concerns" 在实践中很接近

```
total_uncertainty_penalty = min(10, 所有适用 penalty 之和)
```

Lifecycle 减半规则（来自 v2 spec）

如果 Lifecycle 信号显示用户以前订过这个服务 → uncertainty penalty 减半。因为历史数据比自我预测更可信，"说不准"的影响应该更小。

1.9 Final Score Computation（最终分数计算）

```
raw = need_fit + usage + budget_fit + overlap + risk - uncertainty_penalty
total = clamp(raw, 0, 100)
```

Recommendation mapping（推荐结果映射 — 修订版：提高阈值 + 更细粒度）

为什么旧阈值太宽松

旧映射（≥70 = Subscribe Now）有问题：

1. **70 分太容易达到** — 5 个维度总计 100 分，一个给出中等偏上回答的用户就能到 70+（如 need=18, usage=18, budget=14, overlap=10, risk=10 = 70）。这不是一个自信的"立刻订阅"信号。
2. **70 以上无区分** — 72 分和 95 分得到同样的推荐，但信心水平完全不同。
3. **分数挤压** — 实际中大部分诚实用户落在 55-80 区间，<40 和 >85 很少见。推荐区间应该匹配实际分布。

新推荐映射

Score	Uncertainty	Recommendation	理由
≥80	low	✅ **Subscribe Now**	高门槛 — 大部分维度都强
75–79	low	✅ Subscribe Now（柔和版）	强匹配但不压倒性 — verdict 语气略克制
≥75	medium/high	🔍 Start Trial	本该 Subscribe Now，但不确定性使试用更安全
60–74	any	🔍 **Start Trial**	有明确兴趣但不足以全额投入
45–59	any	👀 **Watchlist**	混合信号 — 部分维度强，其他弱
<45	any	🔄 **Consider Alternatives**	大部分维度信号差

"Subscribe Now"（75+）内的细粒度

分数范围	Verdict 语气	模板风格
90–100	热情	"This is as clear a yes as it gets."
80–89	自信	"Solid fit for your current setup."
75–79	克制	"Looks good — just make sure the first month proves it out."

分数曲线（可选，数据驱动）

收集到真实用户数据后（目标：200+ 次 audit），应用归一化曲线防止分数聚集：

```
// Phase 1（无数据阶段）：使用上面的原始分数 + 新阈值
// Phase 2（200+ 次 audit 后）：计算基于百分位的曲线

// 概念方法：
// 1. 收集真实 audit 的分数分布
// 2. 如果 >60% 的分数落在 55–75 区间，说明系统区分度不够
// 3. 用 sigmoid 拉伸中间区域：
//    curved = 50 + 50 × tanh(k × (raw − median) / stddev)
//    k = 调节参数（从 1.0 开始）
// 4. 用 curved 分数重新映射推荐区间
// 5. A/B 测试曲线 vs 原始分数验证用户满意度

// 重要：v3 上线前不实施曲线。
// 当前线性评分 + 提高阈值是 v3 基线。
// 曲线记录在此作为计划中的优化，不是上线要求。
```

验证计划

在生产环境最终确定阈值前：

1. **模拟测试** — 用 50 个合成用户画像（覆盖节俭到高消费、新手到老手、低到高参与度）跑评分引擎。检查分布大致为：15% Subscribe Now, 35% Start Trial, 30% Watchlist, 20% Alternatives。如果严重倾斜则调整阈值。
2. **边缘情况审查** — 手动验证这些场景产生预期推荐：
   - "每天用，预算舒适，无重叠，无风险" → 应该 ≥85
   - "动机模糊，每周一次，预算紧张，有些重叠" → 应该 50–65
   - "冲动购买者，之前因用太少取消过" → 应该 <50
3. **用户反馈循环** — 100 次真实 audit 后，检查收到 "Subscribe Now" 的用户是否真的订阅了并留存 3+ 个月。如果留存率 <60%，阈值太宽松。

Part 2: Conversation Flow（对话流程）

2.1 Stage Map（7 个阶段）

```
1. service_price      → 你在考虑什么服务？多少钱？              [text input]
2. history_check      → 你以前用过这个吗？                      [chips]          ← 新增
3. motivation         → 主要是为了什么？                        [chips + 追问文本]
4. expected_usage     → 你觉得你会多经常用？                    [slider]         ← 新输入形态
5. budget_feel        → 这个价格对你来说怎么样？                [chips]
6. overlap_decision   → 和你现有的订阅重叠吗？                  [chips]
7. independent_risk   → 除了这些，还有什么让你犹豫的？          [chips + 文本框]
```

最少轮次：7（每个 stage 一轮）

最多轮次：~10（包含 history_check 和 motivation 的追问）

2.2 Stage 2: history_check（新增）

这是 v2 spec 里 Lifecycle Signal 的核心采集点，当前代码完全没有实现。

先查记录再提问：引用现有订阅数据

在展示 history_check 问题之前，系统必须先查询用户的订阅数据库，找是否有匹配 `service_name` 的记录（模糊匹配，也检查已知别名如 "YT Premium" → "YouTube Premium"）。

为什么要先查记录？

1. **避免明知故问** — 如果数据库显示用户 3 个月前取消了 Netflix，再问"你用过 Netflix 吗？"显得很蠢
2. **为用户回答提供上下文** — 展示我们已知的信息帮助用户给出更准确的回答
3. **防止矛盾** — 用户可能忘了自己订过；记录可以纠正

预查逻辑

```
// 在展示 Stage 2 问题之前
matchingRecords = querySubscriptions(service_name, user_id)
// 模糊匹配："youtube" 匹配 "YouTube Premium"、"yt premium" 等
```

```
if matchingRecords.length > 0:
  // 找到历史记录 — 调整问题措辞，预填上下文
  mostRecent = matchingRecords.sortByDate().last()
  history_context = {
    had_before: true,
    status: mostRecent.status,        // 'active' | 'cancelled' | 'expired'
    cancel_date: mostRecent.cancelled_at,
    duration: mostRecent.subscribed_duration_months,
    usage_rate: mostRecent.avg_check_in_satisfaction,  // 来自 Check-in 数据（如果有）
  }
else:
  history_context = { had_before: false }
```

问题措辞（根据预查结果调整）

预查结果	问题措辞	理由
找到记录：已取消	"I see you had {service_name} before — you cancelled about {N} months ago. What's bringing you back?"	引用已知历史，体现系统的智能感
找到记录：活跃中	"Looks like you already have {service_name}. Are you thinking about upgrading or changing your plan?"	避免冗余问题，直接跳到相关追问
找到记录：试用期到期	"You tried {service_name}'s free trial back in {month}. Did it feel like it was enough, or did you hit limits?"	用具体上下文问出更好的追问
无记录	"Have you used or subscribed to {service_name} before?"	原始通用问题 — 无数据时适用

Options（无记录时显示，或用于确认/修正记录推断）

Label	value	后续动作
Never tried it	`first_time`	无追问，进入 Stage 3
Tried it, cancelled	`cancelled`	追问 "What made you cancel?" → price / low_usage / found_alternative / other
Currently using, considering upgrade	`upgrading`	**跳过 Stage 4**（usage 直接给 23 分），进入 Stage 5
Used a free trial	`free_trial`	追问 "Did the free version feel limiting?" → yes/no

修正机制

找到记录时，系统根据记录预填 `history_status`，但仍显示选项让用户可以修正：

```
// 如果记录显示已取消，预选 "Tried it, cancelled" chip
// 但用户可以点其他 chip 来覆盖
// 用户的明确选择始终覆盖数据库推断
```

采集的 Slots

```
history_status: 'first_time' | 'cancelled' | 'upgrading' | 'free_trial'
cancel_reason?: 'price' | 'low_usage' | 'found_alternative' | 'other'
free_trial_limiting?: boolean
history_source: 'database' | 'user_reported'  // 新增：记录信息来源
previous_duration_months?: number              // 新增：来自数据库记录
```

开场白路径分叉（来自 v2 spec）

条件	开场白
无历史（first_time），无记录	"What's bringing you to consider this?"
找到记录：已取消	"Looks like you've been here before — what's pulling you back?"
记录 + 取消原因 = low_usage	"Last time was {N} months and it didn't quite stick — what's different now?"
记录 + 取消原因 = price	"Price was the issue last time. Let's see if the math works better now."
用户自报取消（无记录）	"Got it — what made you cancel back then?"

Claude prompt 规则：保持措辞一致性

```
RULE: 如果 history_context.had_before 为 true，你必须在问题中自然地
引用数据库记录。绝不能在我们已有记录证明用户用过的情况下问
"have you used this before?"。

如果用户修正了记录（如记录显示取消但用户说"其实我没订过，
是别人用了我的账号"），确认修正：
```

```
"Ah, my mistake — thanks for clarifying."
然后更新 history_status 为用户的选择。
```

2.3 Stage 3: motivation（带追问）

Options

Label	value
Work / study	work
Entertainment	entertainment
Health & fitness	health
Learning	learning
Social / communication	social
Creative / design	creative
Other	other

注意：新增了 "Social / communication" 和 "Creative / design"，覆盖 v1 的盲区。

追问触发规则（来自 v2 spec Part 6）

如果 Claude 把 `motivation_clarity` 分类为 `category_only`（用户只选了 chip 没有补充，或给了泛泛的描述）：

> "What kind of {motivation} tasks are you picturing this for?"

追问只触发一次，不强制回答。用户回答后，Claude **重新分类** `motivation_clarity`（基于原始回答 + 追问回答的综合内容）：

重新分类为	对 Need Fit base score 的影响
`scenario_specific`（用户描述了具体场景）	升级到 22
`problem_aware`（用户指出了缺口/痛点）	升级到 18
仍然是 `category_only`（用户又给了一个模糊回答）	保持 14

注意：这里用的是 §1.2 中相同的 Claude 语义分类——**不涉及字数统计**。追问的作用是给用户第二次机会来更具体地表达需求。

2.4 Stage 4: expected_usage（Slider 输入）

视觉



用 Slider 的好处：打破连续点 chip 的单调感，增加物理交互的参与感。

映射

Slider 范围	显示标签	映射值
0–10%	Rarely	0
10–25%	About once a week	1
25–50%	A few times a week	2.5
50–75%	Most days	4.5
75–100%	Daily	7

动态问题措辞

Claude 根据 `service_name` 调整提问：

- 流媒体/娱乐类："How many days per week would you actually watch/listen?"
- 工具/软件类："How often per week would you open this?"
- 健身/健康类："How many sessions per week realistically?"
- 默认："How often would you realistically use this per week?"

2.5 Stage 5: budget_feel（带上下文）

问题（增强版）

不再是冷冰冰的 "How does the price feel?"，而是 Claude 生成带数字的版本：

> "At $10.99/month — that's about 8% of your subscription budget — how does that feel?"

百分比在客户端算好，注入到 prompt context。让用户带着具体数字思考。

Options（不变）

Label	value
Comfortable	comfortable
A stretch	tight
Too much	no_way
Not sure	not_sure

2.6 Stage 6: overlap_decision（引用现有订阅名称）

问题（增强版）

如果用户的订阅列表里有同类别的服务，Claude 必须点名：

> "You currently have Spotify and YouTube Premium. How much does Apple Music overlap with those?"

如果没有同类别订阅：

> "Does this overlap with anything you're already paying for?"

数据已经在 `subscriptions` context 里了，只需要在 prompt 里加规则。

Options（不变）

2.7 Stage 7: independent_risk（重新设计）

Question

> "Besides what we've already talked about — is there anything that makes you hesitate?"

Options

Label	value
No real concerns	none
The free version might be enough	free_enough
I might forget to cancel	forget_cancel
I tend to lose interest quickly	interest_decay
I'm being impulsive right now	impulse
Not sure	not_sure

始终显示文本输入框

Chips 下方永远有一个文本框，用户可以点 chip 也可以写自己的顾虑。自定义文本存为 `additional_risk_note`，报告里会引用。

2.8 Mid-Conversation Progress Feedback（中途总结反馈）

在 Stage 4 和 Stage 5 之间，Bot 插入一条总结：

> "Okay, here's what I have so far — {service_name}, ${price}/month, mainly for {motivation}. A few more questions and I'll have your score ready."

三个作用

1. 用户确认信息是否正确（纠错机会）
2. 用户知道快结束了（减少疲劳感）

3. 打破 问→答→问→答 的单调节奏

实现方式

这是一条非问题气泡，不需要用户回应。显示后等 1.5 秒，然后接 Stage 5 的问题。

2.9 Interaction Design: 让对话有"被听到"的感觉

Echo Back（回应感）

Claude 的 `bubbles`（问题前面的上下文内容）必须先用 1-3 个词确认用户的上一条回答：

```
规则：bubbles 的第一句必须简短地 acknowledge 用户的回答。示例：
- "Makes sense."
- "Got it."
- "Fair enough."
- "$10.99/month — noted."
- "Interesting choice."

然后再在同一条或第二条 bubble 里提供 context/insight。
不允许跳过 acknowledgment。
```

Conditional Warmth（条件性幽默/温度）

```
规则：
- 如果用户的回答是正面的（comfortable / none / daily），可以加一句轻松的话：
  "Not everyone can say that about their subscriptions."
  "Clean slate — that makes this easier."

- 如果用户的回答是负面或脆弱的（no_way / impulse / interest_decay），
  保持尊重中性，不开玩笑：
  "Honest answer. That's actually really useful."
  "Good that you know that about yourself."
```

Input Type Variation（输入形态变化）

Stage	输入形态	为什么
1	Text input	需要自由描述
2	Chips (4)	简单分类
3	Chips (7) + 追问 text	先分类再细化
4	**Slider**	视觉新鲜感，打破 chip 单调
5	Chips (4)	快速判断
6	Chips (5)	快速判断
7	Chips (6) + text input 始终可见	允许自定义回答

Part 3: Aha Moment System（Aha 系统）

3.1 两个场景：Conversation Aha vs Report Watch Items

Aha 的本质是一个**底层信号**（比如 `portfolio_overlap_high`），不是一个显示元素。同一个信号在对话和报告中用不同的"语态"表达。

展示层	语态	功能	格式
对话中	事实陈述（告诉你一个事实）	影响用户后续回答的思考	💡 卡片（独立视觉，不是聊天气泡）
报告 Why	分析叙述（把事实编进逻辑链）	解释为什么是这个分数	融入 2-3 句自然语言段落
报告 Watch Items	行动引导（告诉你该做什么）	给具体的下一步	💡 图标 + 行动建议

去重规则

```
报告 Watch Items =
  所有风险信号
  减去 对话中已经用 Aha 展示过的事实性内容
  加上 所有信号的行动建议
```

核心原则：对话说事实，Why 说影响，Steps 说行动——各司其职，零重复。

以 `portfolio_overlap_high` 信号为例：

展示层	内容
对话 Aha	"You already have Apple Music and YouTube Premium."（事实）
报告 Why	"...with two music services already in rotation, adding a third is a pattern worth watching."（分析——不重复事实）
报告 Watch Items	"Try going 2 weeks without Apple Music. If you don't miss it, swap instead of stack."（行动——不重复事实和分析）

3.2 Conversation Aha: 视觉设计

Aha 是一张**插入对话流中的卡片**，和普通聊天气泡视觉明显不同：

```
┌ - - - - - - - - - - - - - - - - - - ┐
│ 💡                                  │
│ You already have Apple Music and    │
│ YouTube Premium. That's 3 music     │
│ services if you add this one.       │
└ - - - - - - - - - - - - - - - - - - ┘
```

属性	值	和普通气泡的区别
背景色	`colors.warning + '10'`（淡暖黄）	普通气泡是白色
边框	1px dashed `colors.warning + '40'`	普通气泡是 solid gray
宽度	全宽（两侧小边距）	普通气泡左对齐，最大 85%
图标	💡（灯泡）	普通气泡用 Bot avatar
文字风格	`fontSize.sm`，italic	普通气泡正常字号
最大长度	2 句，≤30 词	—

出现时机

Aha 卡片在**触发条件被满足的那条用户回答之后**立刻出现，在下一个问题之前：

```
用户回答 → 600ms 停顿 → 💡 Aha 卡片 → 700ms 停顿 → 下一个问题
```

频率限制

每次 Audit **最多 1 次**对话 Aha。保持稀缺性——如果每隔一轮就出现，用户会麻木。

3.3 Conversation Aha: 触发条件（扩展到 10 种）

#	触发条件	Aha 内容方向	优先级
1	history_check = cancelled + cancel_reason = low_usage	"Last time didn't stick — what's genuinely different now?"	HIGH
2	overlap_level = significant/high 且系统检测到具体重叠的订阅名	"You already have {sub1} and {sub2}. That's {N} {category} services if you add this."	HIGH
3	budget_feel = comfortable 但 ratio > 15%	"Good that it feels right — just flagging it's {X}% of your monthly budget."	MEDIUM
4	expected_frequency = daily 但同类别订阅的 check-in 使用率在下降	"You said daily — but your other {category} subs have been trending down lately."	HIGH
5	risk = impulse	"Knowing that about yourself is actually a good sign. The trial period is your safety net."	MEDIUM
6	矛盾检测（如 budget_feel = no_way 但动机很强）	"Sounds like you really want it but the timing isn't right."	HIGH
7	expected_frequency ≤ 1 且 price_monthly > 15	"At *price/monthforroughlyfrequseperweek—that's* {cost_per_use} each time."	MEDIUM
8	history_check = cancelled +	"You found something better last time. Is that alternative still working for you?"	MEDIUM

#	触发条件	Aha 内容方向	优先级
	cancel_reason = found_alternative		
9	Portfolio Pattern 触发（同类别 ≥2，avg usage <0.4）	"You have {N} {category} subscriptions. Are you actually rotating between them?"	HIGH
10	用户给了详细 motivation_note 且 persona 强匹配	"For someone who {persona_trait}, that makes a lot of sense."（正面 Aha）	LOW

选择逻辑

如果多个条件同时触发，选优先级最高的。同级时选最早触发的（离相关上下文更近）。

3.4 Tracking Triggered Insights（追踪已触发的信号）

```
// 对话过程中记录
const [triggeredInsights, setTriggeredInsights] = useState<string[]>([]);

// 当 Aha 触发时
setTriggeredInsights(prev => [...prev, 'portfolio_overlap_high']);

// 生成报告时传入
generateReport(slots, scoreResult, triggeredInsights);
```

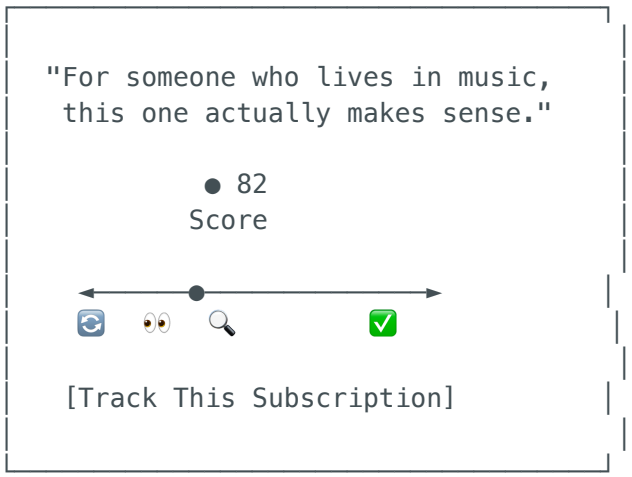
报告的各层生成器检查 `triggeredInsights`，避免重复已经说过的事实。

Part 4: Report Output（报告输出）

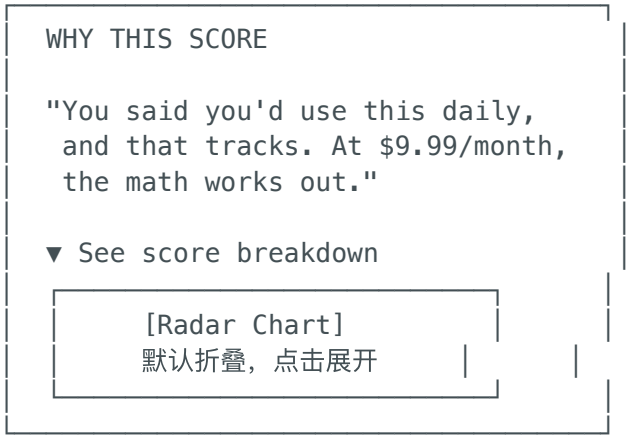
4.1 Three-Layer Structure with Progressive Disclosure（三层渐进式展示）

设计原则：**80% 的用户只需要看第一屏就够了。** 越往下滑，信息越深，但只有想深挖的用户才需要。

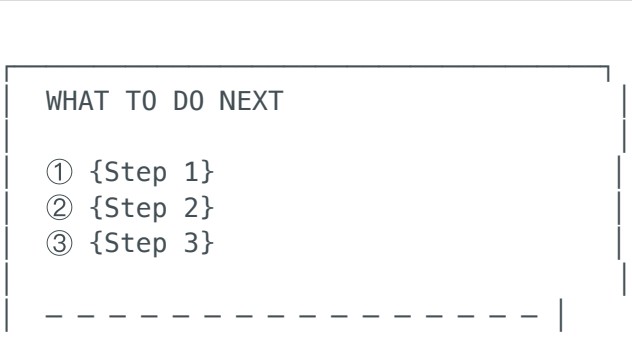
第一屏（3 秒读完，不需要滑动）



第二屏（往下滑，30 秒阅读）



第三屏（继续往下滑，行动导向）



```
│                                          │
│  💡 Watch for:                           │
│  → {Watch item 1}                        │
│  → {Watch item 2}                        │
│                                          │
│  [Start Another Audit]                   │
└──────────────────────────────────────────┘
```

4.2 Layer 1: Verdict

生成方式

规则选模板 → Claude 在约束内润色。

模板选择

条件	模板风格	示例
Score ≥80 + persona 强匹配	引用身份认同	"For someone who lives in music, this one actually makes sense."
Score 75–79 + persona 强匹配	克制自信	"Your setup says yes — just make sure the first month confirms it."
Score ≥75 + 无 persona 关联	干净自信	"Solid fit for your current setup."
Score 60–74	承认张力	"Could work, but worth a trial first."
Score 60–74 + portfolio pattern	反思	"Not a bad pick — but you're stacking in {category}."
Score 45–59	温和重定向	"Not quite ready. Worth saving for later."
Score <45 + free_enough	具体替代建议	"The free version might actually cover what you need."
Score <45 + 通用	诚实但善意	"Probably not the right time for this one."

约束

```
- 最多 15 个词
- 必须是一个完整的句子
- 不许以 "Based on" 开头
- 不许用 "subscription" 这个词
```

Score Bar（分数标尺）

Verdict 下方显示一个水平标尺，让用户直观看到自己在哪个区间：

```
0──────45──────60──────75──────100
  🔄        👀        🔍        ✅
                        ●82
```

用户一看就知道"我在绿色区间"，不需要理解阈值。（注意阈值已上调：75+ 才是 Subscribe Now）

4.3 Layer 2: Why（Pro/Con 格式，对齐雷达图维度）

相比之前草案改了什么

之前用一段 2-3 句的自然语言。团队反馈：

1. **一长句很难解析** — 用户需要反复阅读才能分清什么是好的、什么是不好的
2. **和雷达图没有对应关系** — 段落讨论信号但不映射到用户刚看到的可视化维度
3. **Pro/Con 更可扫描** — 用户瞬间看到"2 个优势，1 个劣势"

新格式：结构化 Pro/Con

```
WHY THIS SCORE

✅ What's working:（优势）
• {pro_1 — 对应雷达图最高维度}
• {pro_2 — 对应雷达图次高维度}

⚠️ What's not:（劣势）
• {con_1 — 对应雷达图最低维度}
```

```
💬 {一句话总结 — "so what"}
```

生成方式

Pro/Con 条目是规则生成的（来自雷达图数据），**总结语句由 Claude 生成**（保证自然语言质量）。

第一步：从雷达数据选取 pros 和 cons

```
// 按百分比分数排序维度
sorted = radarData.sortByPercentage()

// Pros：得分 ≥60% 的前 2–3 个维度
pros = sorted.filter(d => d.percentage >= 60).slice(0, 3)

// Cons：得分 <50% 的后 1–2 个维度
cons = sorted.filter(d => d.percentage < 50).slice(0, 2)

// 如果没有 cons（全 ≥50%）：显示 "Nothing major" + 说明
// 如果没有 pros（全 <60%）：显示 "Limited upside" + 解释
```

第二步：每个维度映射到人话版 pro/con 句子

维度	高分（≥60%）→ Pro	低分（<50%）→ Con
Need Clarity	"You have a clear, specific reason for wanting this" / "你有一个清晰具体的使用理由"	"The reason you want this is still vague — that usually means the need isn't urgent" / "想要的理由还比较模糊——通常意味着需求不紧急"
Expected Use	"You'll likely use this {frequency} — that's solid value" / "你可能会每{frequency}用——这是实在的价值"	"Usage might be too low to justify the cost" / "使用频率可能太低，不足以证明花费合理"
Budget Comfort	"The price fits comfortably in your budget" / "价格在你的预算内很舒适"	"This will stretch your budget — it's {ratio}% of your monthly allocation" / "这会拉紧你的预算——占月预算的{ratio}%"
Uniqueness	"Nothing in your current setup does what this does" / "你目前没有能替代这个的东西"	"You already have tools that overlap with this" / "你已经有功能重叠的工具了"

维度	高分（≥60%）→ Pro	低分（<50%）→ Con
Risk Level	"No major red flags or hesitations" / "没有重大红旗或犹豫"	"You flagged {risk_type} as a concern — that's worth taking seriously" / "你提到了 {risk_type} 的顾虑——这值得认真对待"

第三步：Claude 生成总结语句

```
INPUT:
- Pros: {pro_dimensions and sentences}
- Cons: {con_dimensions and sentences}
- Score: {score}
- Recommendation: {recommendation_type}
- Persona match: {persona_alignment}
- User's own words: {motivation_note}
- History: {history_status}
- Triggered insights: {triggeredInsights}

RULES（总结语句规则）:
- 一句话，最多 25 词
- 把 pros 和 cons 连成一条 verdict 级别的洞察
- persona_alignment = "strong" 时：引用身份
- 如果信号矛盾：简洁地点出张力
- 不重复 pro/con 条目的事实 — 综合它们
- 不重复对话 Aha 中已展示的事实
- 不以 "Based on" 开头
- 不用 "subscription"
- 禁用短语：见 Part 4.6

OUTPUT: 一句话，前缀 💬 。只输出这句话。
```

各场景示例

A. 高分 (82) + persona 强匹配：

```
✅ What's working:
• You have a clear, specific reason for wanting this
• You'll likely use this daily — that's solid value
• The price fits comfortably in your budget

⚠️ What's not:
• (Nothing major — all dimensions scored well)

💬 For someone who lives in music, this is about as clean a fit as it gets.
```

B. 中等分 (63) + 预算紧张：

```
✅ What's working:
• You have a clear use case for this
• Nothing in your current setup does what this does

⚠️ What's not:
• This will stretch your budget — it's 18% of your monthly allocation

💬 The intent is real, but the price might force a trade-off this month.
```

C. 低分 (38) + Portfolio Pattern：

```
✅ What's working:
• (Limited — no dimension scored above 60%)

⚠️ What's not:
• You already have 2 services that overlap with this
• Usage might be too low to justify the cost

💬 Before adding another music service, figure out which existing one you'd
actually drop.
```

D. 中等分 (58) + 高不确定性：

```
✅ What's working:
• No major red flags or hesitations

⚠️ What's not:
• The reason you want this is still vague
• Usage might be too low to justify the cost

💬 There are too many blanks to give a confident answer — a trial would reveal
more than guessing.
```

E. 高分 (78) + 免费版风险：

```
✅ What's working:
• You'll likely use this most days — strong habit signal
• The price fits your budget without strain

⚠️ What's not:
• You mentioned the free version might be enough — that's worth testing first

💬 Everything checks out except one question: are you paying for features or
convenience?
```

这个板块的可读性规则

元素	最大长度	格式
每条 pro/con	20 词	一句话，无子弹点
总结语句	25 词	一句话，以 💬 开头
总共 pros	2-3 条	只选 ≥60% 的维度
总共 cons	0-2 条	只选 <50% 的维度
不出现数字	是	条目里无百分比或分数——那些在雷达图里

和雷达图的关系

Why 板块和雷达图是**同一组数据的两种视图**：

```
雷达图 = 定量视图（百分比、形状、红点）
Why 板块 = 定性视图（人话版 pros 和 cons）

想看数字的用户 → 展开雷达图
想看文字的用户 → 读 Why 板块
两者引用同样的 5 个维度，互相印证
```

4.4 Radar Chart（雷达图，在 "See Breakdown" 折叠区域内）

五个轴

内部名	显示名
need_fit	Need Clarity
usage	Expected Use
budget_fit	Budget Comfort
overlap	Uniqueness
independent_risk	Risk Level

数据映射

每个维度分数转为 0-100% 百分比：

```
radarData = {
  needClarity:   (need_fit / 25) * 100,
  expectedUse:   (usage / 25) * 100,
  budgetComfort: (budget_fit / 20) * 100,
  uniqueness:    (overlap / 15) * 100,
  riskLevel:     (risk / 15) * 100,
}
```

视觉设计

- **填充颜色：**
  - 总分 ≥70：`colors.primary`（蓝色）20% opacity
  - 总分 50–69：`colors.warning`（橙色）20% opacity
  - 总分 <50：`colors.danger`（红色）20% opacity
- **轴端点：** 显示维度名 + 百分比
- **低分高亮：** 低于 50% 的维度用红色圆点标记 → 用户一眼看到"问题出在哪里"
- **网格线：** 3 个同心五边形，分别在 33%、66%、100% 处
- **实现方式：** `react-native-svg`（项目已有依赖），手绘 5 轴雷达

默认状态

折叠。 用户点 "See score breakdown" 才展开。大部分用户读完 Why 的 2-3 句话就理解了，不需要看详细分数。

4.5 Layer 3: What To Do Next（行动路径）

结构

每份报告都有 **3 个行动步骤**（按推荐类型定制）+ **0-2 个 Watch Items**（条件触发）。

按推荐类型的行动步骤

导航 UX：用户点击 action button 后发生什么？

问题

Action steps 如 "Track in SubSense" 或 "[See 3 Alternatives]" 暗示了页面跳转。如果不定义跳转去哪和怎么回来，UX 就断了：

- 点 "Track in SubSense" → 跳转到订阅追踪页 → 怎么回到报告?
- 点 "See 3 Alternatives" → 跳转到替代品列表 → 浏览完了 → 然后呢?
- 点 "Start the free trial" → 这甚至不是 app 内操作——是外部链接

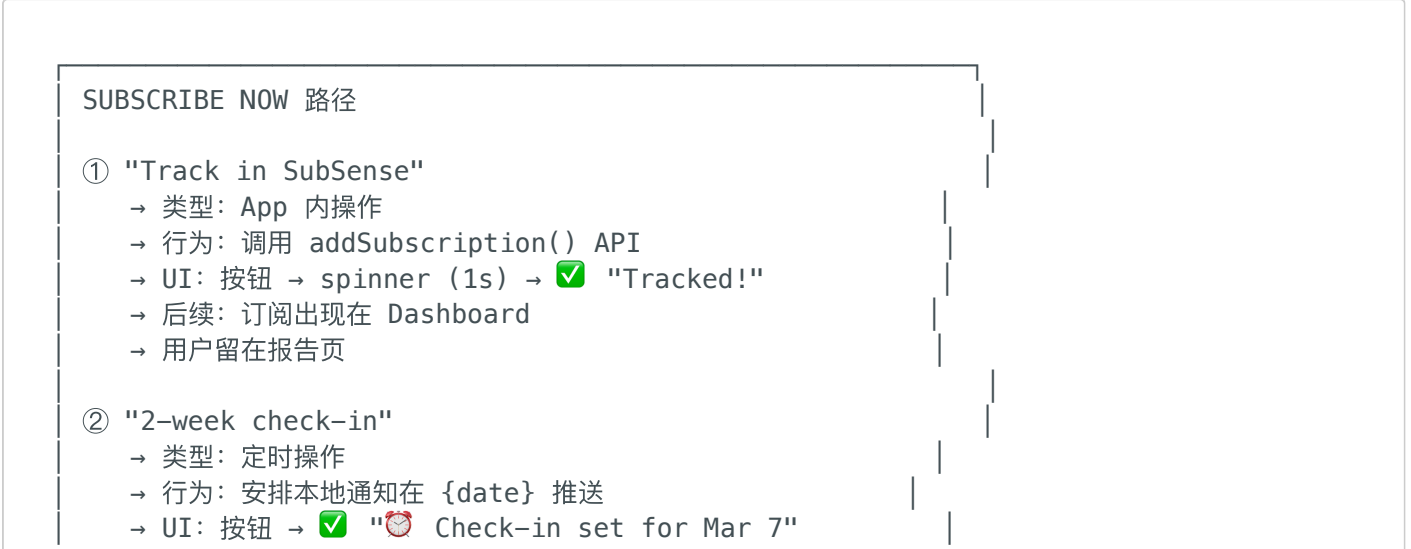
设计原则：Audit 报告是锚点

报告是一个**持久化的产物**，用户可以随时回来。所有 action step 的导航以报告为锚点。

Action 类型和导航流程

Action 类型	行为	导航模式	返回行为
App 内操作（Track, Watchlist, Set reminder）	在报告页内即时完成，无需跳转	**行内确认** — 按钮变成 ✅ "Done"，留在报告页	不适用——用户从不离开
App 内导航（See Alternatives, Re-audit）	打开 app 内新页面	**Stack 导航** — push 新页面到导航栈	Back 按钮返回报告，滚动位置保持
外部操作（Start free trial, Visit website）	打开外部 URL 或 App Store	**系统浏览器/深度链接** — 在 Safari 或外部 app 打开	iOS app-switcher 返回 SubSense；报告状态保持
定时操作（2-week check-in, Renewal reminder）	创建本地通知/日历事件	**行内确认** — "⏰ Reminder set for {date}" 出现在步骤下方	不适用——用户从不离开

每个 action step 的详细流程



```
    → 用户留在报告页
③ "Renewal reminder"
    → 类型：定时操作
    → 行为：安排通知在 renewal_date 推送
    → UI：按钮 → ✅ "⏰ Reminder set for Apr 15"
    → 用户留在报告页
```

```
START TRIAL 路径

① "Start the free trial"
    → 类型：外部操作
    → 行为：在 Safari 打开 service_url
    → UI：按钮显示外链图标（↗）
    → 返回：用户通过 iOS 切回 SubSense
    → 报告仍在同一滚动位置

② "Use it {frequency} this week"
    → 类型：信息型（无点击操作）
    → 行为：纯文字提醒，不是按钮

③ "Decision point on Day {N}"
    → 类型：定时操作
    → 行为：安排通知在 trial_end 推送
    → UI：按钮 → ✅ "⏰ Decision reminder set"
```

```
ALTERNATIVES 路径

① "See 3 Alternatives"
    → 类型：App 内导航
    → 行为：Push AlternativesScreen 到导航栈
    → 页面展示：3 个替代品 + 名称、价格、差异点、
      "Audit this instead" 按钮
    → 返回：Back 按钮（←）回到报告
    → React Navigation stack 自动保持报告滚动位置

② "Ask yourself: {question}"
    → 类型：信息型（无点击操作）
    → 行为：纯反思文字

③ "Audit again"
    → 类型：App 内导航
    → 行为：Push 新 AuditScreen 到导航栈
    → 预填：service_name（同一服务）
    → 之前的报告留在栈里——用户可以返回查看
```

Action 类型的视觉区分

Action 类型	按钮风格	图标
App 内操作（即时）	Primary 实心按钮（蓝色）	无
App 内导航	线框按钮（蓝色边框）	→ 箭头

Action 类型	按钮风格	图标
外部操作	线框按钮（灰色边框）	↗ 外链
定时操作	Primary 实心按钮（蓝色）	⏰ 钟表
信息型（无操作）	无按钮——纯文字 + 步骤编号	💡 提示图标

状态保持

```
// 报告页面导航离开后保持状态
// 利用 React Navigation 内建的 stack 状态保持：

// AuditReportScreen 保存：
// - 滚动位置（通过 ScrollView ref）
// - 哪些 action button 已点击（local state）
// - 雷达图展开/折叠状态

// 用户导航到 AlternativesScreen 或新 AuditScreen
// 按 back 后，所有状态自动恢复
```

"Start Another Audit" 按钮（报告最底部）

报告最底部的这个按钮：

- **重置** audit 流程（新服务，全新对话）
- **替换**导航栈中的当前报告（不是 push——防止栈无限增长）
- 之前的报告保存到历史记录，可从 Audit History 列表访问

按推荐类型的行动步骤

Subscribe Now (≥75, uncertainty low)

```
① Track {service_name} in SubSense
   → So you never lose sight of renewals

② 2-week check-in on {date}
   → Quick "still using it?" ping to confirm the habit stuck

③ Renewal reminder on {renewal_date}
   → We'll remind you 3 days before {billing_cycle} renewal
```

Start Trial (60–74)

```
① Start the free trial
   → Most services offer 7–14 days — avoid entering payment info if possible

② Use it {expected_frequency_text} this week
   → You said {frequency} — let's see if that holds up in practice

③ Decision point on Day {trial_end}
   → We'll ping you before it converts: keep or cancel?
```

Watchlist (45–59)

```
① Saved to your watchlist
   → It's not going anywhere. Revisit anytime.

② Before you come back, figure out:
   → {dynamic_question_1}
   → {dynamic_question_2}

③ Re-audit in 30 days
   → We'll nudge you to take another look with fresh eyes
```

Consider Alternatives (<45)

```
① Check these out first
   → [See 3 Alternatives] — at least one is free

② If none fit, ask yourself:
   → {dynamic_reflection_question}

③ Still want it?
   → Come back and audit again — your score updates with new answers
```

Dynamic Questions（动态问题，Watchlist 专用）

从下表中选最相关的 top 2：

条件	问题
Motivation 模糊（只选 chip 没补充）	"Can you describe a specific moment this week where you needed {service_name} and didn't have it?"
budget_feel = tight 或 no_way	"If your budget freed up $20 next month, would this be the first thing you'd spend it on?"

条件	问题
overlap_level = some	"Have you tried using {existing_similar_sub} for the same purpose? What's actually missing?"
expected_frequency ≤ 1	"If you had it right now, what would you use it for today — literally today?"
history_check = cancelled	"What's specifically different in your life now vs. last time you had this?"
无特定条件匹配	"Sleep on it for a week. If you're still thinking about it, that's a real signal."

Watch Items（条件触发，最多 2 条）

选取逻辑：

```
candidateWatchItems = 所有匹配条件的 items
过滤掉 事实已在对话 Aha 中展示过的（但保留行动版本）
按严重度排序（HIGH first）
取 top 2
```

Watch Item 模板：

条件	Watch Item（行动导向，不重复事实）
history_check = cancelled, reason = low_usage	"Watch whether the old pattern repeats. If you skip a full week, that's your signal to cancel."
history_check = cancelled, reason = price	"Make sure the trial auto-renews at a price you're okay with. Set a reminder for the conversion date."
expected_frequency ≤ 1	"If you find yourself skipping weeks, that's your signal. Don't wait for the renewal to decide."
budget_feel = tight	"If anything else comes up this month financially, this is probably the first cut. Plan for that."
overlap_level = some/significant	"Try going a week without {overlap_service} first. If you don't miss it, swap instead of stack."
risk = impulse	"You flagged this as impulsive — the trial period is your safety net. Use it fully before committing."

条件	Watch Item（行动导向，不重复事实）
risk = interest_decay	"Set a 2-week checkpoint: if you haven't used it 3+ times by then, cancel without guilt."
risk = forget_cancel	"Auto-renewal is the trap. Set a calendar reminder for {cancel_deadline_date} right now — literally right now."
risk = free_enough	"Max out the free tier first. If you hit a wall, that's your proof the upgrade is worth it."
not_sure_count ≥ 2	"You weren't sure about a few things — that's fine, but get clarity on {unclear_slots} before committing real money."
Portfolio Pattern 触发	"Before adding another {category} service, figure out which existing one you'd drop. If none — that's your answer."

4.6 Readability Rules（可读性规则）

渐进式展示

层	默认状态	内容
Verdict + Score + CTA	**始终可见**（第一屏）	3 秒决策
Why 段落	滑动可见	30 秒理解
雷达图 + 维度明细	**折叠**（点击展开）	给想深挖的用户
Next Steps + Watch Items	滑动可见	行动指南

文案长度硬限制

内容	最多词数	格式
Verdict	15	一个句子
Why	60	2-3 句，纯叙述，不用 bullet
每条 Watch Item	30	两句话：上下文 + 行动
每个 Step	10	动词开头的祈使句

Why 段落里不出现分数数字

所有数字在雷达图里展示。文字和数字各管各的——混在一起两边都更难处理。

Banned AI Phrases（禁用的 AI 腔短语）

```
NEVER use:
- "Based on your/our assessment/analysis"
- "Aligns with your preferences"
- "Taking into account"
- "It's worth noting that"
- "Comprehensive"
- "Optimize your subscription portfolio"
- "Make an informed decision"
- "I recommend considering"
- "Let's dive in"
- "Here's the thing"

Write like texting a smart friend, not drafting a consulting report.
像给聪明朋友发消息那样写，不要像在写咨询报告。
```

Part 5: Claude Prompt Architecture（Claude Prompt 架构）

5.1 Conversation System Prompt（对话系统提示词）

```
You are SubSense AI — a friendly, sharp subscription advisor.
You help users decide whether to subscribe to a new service
through a structured 7-stage conversation.

## CURRENT SESSION STATE
User: {user_name}
Monthly budget: ${monthly_budget}
Existing subscriptions: {existing_subs_list}
Current stage: {stage}
Turn index: {turn_index}
Collected slots: {slots_json}
History check result: {history_status}
Persona tags: {interest_tags}
Subscriber type: {subscriber_type}

## STAGE MAP (follow in order, never skip unless noted)
Stage 1 — service_price: Ask what service + price/billing cycle.
Stage 2 — history_check: Ask if they've used this before.
Stage 3 — motivation: Ask primary reason. If user only selects chip
  without elaboration, follow up ONCE asking for specific use case.
```

```
Stage 4 — expected_usage: Ask realistic usage frequency.
  Adapt unit to service type (days/week for streaming, times/month
  for tools, sessions/week for fitness).
  SKIP this stage if history_check = "upgrading".
Stage 5 — budget_feel: Reference price as % of budget in the question.
  If existing_subscriptions has same-category items, mention total
  category spend.
Stage 6 — overlap_decision: If same-category subs exist, name them
  in the question. Otherwise ask generically.
Stage 7 — independent_risk: Ask "Besides what we've talked about,
  is there anything that makes you hesitate?"

## CONVERSATION STYLE
- First bubble must acknowledge user's previous answer in 1-3 words
  ("Got it.", "Makes sense.", "$10.99 — noted.")
- If user's answer is positive: may add one warm/light remark
- If user's answer is negative/vulnerable: stay respectful, never joke
- Questions are curious and specific, never clinical
- Maximum 2 bubbles (context) + 1 question per turn

## OUTPUT FORMAT
Return ONLY valid JSON. Schema:
{
  "stage": "string",
  "turn_index": number,
  "is_finished": false,
  "bubbles": ["string"],
  "question": "string",
  "ui": {
    "input_mode": "options" | "text" | "slider",
    "allow_optional_note": boolean,
    "note_hint": "string",
    "max_chars": number
  },
  "options": [{"id","label","value"}],
  "slots_delta": {},
  "missing_slots": [],
  "flags": {
    "contradiction": boolean,
    "off_topic": boolean,
    "price_cycle_unclear": boolean
  },
  "aha": {
    "triggered": boolean,
    "bubbles": ["string"]
  }
}
```

5.2 Report Why Prompt（报告 Why 总结语句生成提示词 — 只生成总结句）

注意：Why 板块的 pro/con 条目是**规则生成的**（来自雷达图数据，见 §4.3）。Claude 只生成底部的**总结语句**（💬）。

```
You are writing the synthesis sentence for a subscription audit's
"Why This Score" section. Pro/con bullets have already been shown
above this sentence. Your job: ONE sentence that ties them together.
你正在为订阅审核的 "Why This Score" 板块写总结语句。
pro/con 条目已经展示在上方。你的任务：一句话把它们串起来。

INPUT:
- Pros shown to user: {pro_sentences}
- Cons shown to user: {con_sentences}
- Score: {score}
- Recommendation: {recommendation_type}
- Persona match: {persona_alignment}
- Persona tags: {tags}
- Service: {service_name}
- User's own words: {motivation_note}
- History: {history_status}
- Conversation Aha already shown: {triggeredInsights}

RULES:
- ONE sentence, maximum 25 words / 一句话，最多 25 词
- Connect the pros and cons into a single verdict-level insight
  把 pros 和 cons 综合成一条 verdict 级别的洞察
- If persona_alignment = "strong": reference identity / 引用身份
- If signals conflict: name the tension concisely / 简洁点出张力
- Do NOT repeat facts from the pro/con bullets — synthesize them
  不重复 pro/con 条目的事实——综合它们
- Do NOT repeat facts shown in conversation Aha
- Never start with "Based on"
- Never use "subscription"
- Banned phrases: [see list in Part 4.6]

OUTPUT: One sentence prefixed with 💬. Nothing else.
只输出一句话，前缀 💬。
```

5.3 Report Verdict Prompt（报告 Verdict 生成提示词）

```
You are writing a one-line verdict for a subscription audit.

INPUT:
- Score: {score}
- Recommendation: {recommendation_type}
- Persona match: {persona_alignment}
- Persona tags: {tags}
- Service: {service_name}
- Top risk signal: {top_risk}

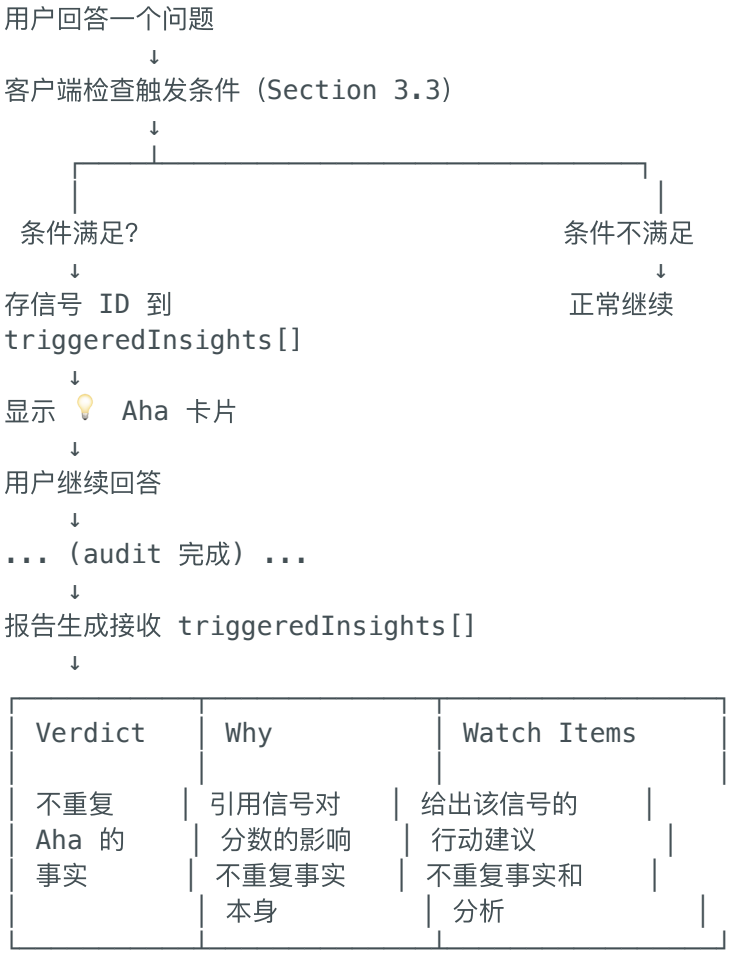
RULES:
- ONE sentence, maximum 15 words
- If persona_alignment = "strong" and score ≥80: use "For someone who..." format
- If score ≥75 and neutral persona: confident and clean
- If score 75–79: measured confidence, slightly less enthusiastic than 80+
- If score 60–74: acknowledge tension
```

```
- If score <45 and risk = free_enough: mention free version
- Never use the word "subscription"
- Never start with "Based on"

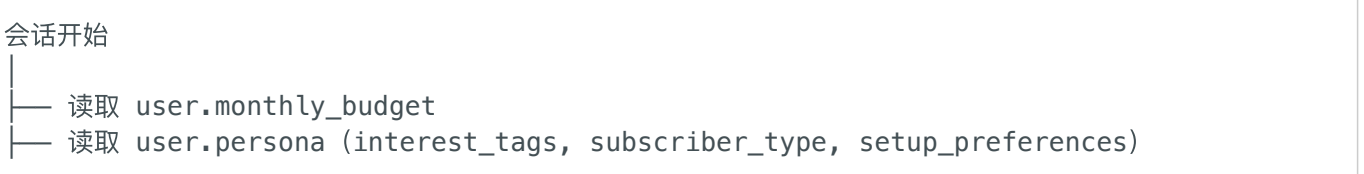
OUTPUT: One sentence. Nothing else.
```

Part 6: Signal Lifecycle & Data Flow（信号生命周期和数据流）

6.1 Aha Signal Flow（Aha 信号流转图）



6.2 Complete Audit Data Flow（完整数据流）



```
├── 读取 user.subscriptions（names, costs, categories, status）
├── 读取 subscription_summaries（来自 Check-in 的 Layer 1，如果有的话）
│
▼
对话（7 个阶段）
│
├── Stage 1: service_price → slots: service_name, price, billing_cycle
├── Stage 2: history_check → slots: history_status, cancel_reason
│   └── 副作用：开场白路径分叉
├── Stage 3: motivation → slots: primary_motivation, motivation_note
│   └── 副作用：可能触发追问
├── [中途总结反馈]
├── Stage 4: expected_usage → slots: expected_frequency
│   └── 副作用：upgrading 时跳过
├── Stage 5: budget_feel → slots: budget_feel
├── Stage 6: overlap_decision → slots: overlap_level
├── Stage 7: independent_risk → slots: top_risk, additional_risk_note
├── [最终补充窗口："Anything else?"]
│
├── 全程：Aha 系统监控 → triggeredInsights[]
│
▼
评分引擎
│
├── calculateNeedFit(slots, personaAlignment) → 0–25
├── calculateExpectedUsage(slots, lifecycleSignal) → 0–25
│   └── 与 Need Fit 交叉验证
├── calculateBudgetFit(slots, monthly_budget) → 0–20
│   └── Cost-per-use 微调
├── calculateOverlap(slots, portfolioPattern) → 0–15
├── calculateIndependentRisk(slots) → 0–15
├── calculateUncertaintyPenalty(slots, lifecycleSignal) → 0 to −10
│
├── raw = 各维度之和 − penalty
├── total = clamp(raw, 0, 100)
├── recommendation_type 由 score + uncertainty 决定
│
▼
报告生成
│
├── Verdict：规则选模板 → Claude 润色（≤15 词）
├── Why：Claude 生成，基于信号 + prompt 规则（≤60 词）
├── Radar chart 数据：每个维度的百分比
├── Next Steps：规则引擎，每种推荐类型 3 步
├── Watch Items：由 triggeredInsights 过滤，取严重度 top 2
│
▼
用户 CTA 行为（写回系统）
│
├── 推荐 "Subscribe Now" → 用户选了 "Watchlist" = 记录 "decision_hesitation"
├── 推荐 "Start Trial" → 用户直接订了 = 记录 "risk_tolerance_high"
├── 推荐 "Alternatives" → 用户无视替代品仍订原服务 = 记录 "brand_preference"
├── 推荐 "Watchlist" → 3 个月后回来 audit = 记录 "slow_decider"
│
└── 行为信号写入 Layer 2 persona（不影响 Layer 1）
```

Part 7: Implementation Plan（实施计划）

文件映射

改动	涉及文件
新评分逻辑（所有维度）	`app/src/utils/auditScorer.ts`（重写）
新 stage 定义 + FSM	`app/src/screens/AuditScreen.tsx`（大幅重构）
新 slots 类型定义	`app/src/types/index.ts`
更新 system prompt + 报告 prompt	`app/src/utils/claude.ts`（重写）
Aha 卡片组件	`app/src/components/features/AhaCard.tsx`（新建）
雷达图组件	`app/src/components/features/RadarChart.tsx`（新建）
Score Bar 组件	`app/src/components/features/ScoreBar.tsx`（新建）
Slider 输入组件	`app/src/components/features/FrequencySlider.tsx`（新建）
报告布局（三层）	`app/src/screens/AuditScreen.tsx` 报告部分
Next Steps + Watch Items 生成器	`app/src/utils/reportBuilder.ts`（新建）
Setup 问题（兴趣标签、订阅者类型）	`app/src/config/setupQuestions.ts`

优先级排序

P0 — 核心逻辑（先做，其他都依赖这个）

1. 更新 `types/index.ts` — 新 slots（history_status, cancel_reason 等）
2. 重写 `auditScorer.ts` — 全部 5 个维度的新逻辑
3. 更新 `claude.ts` — 新 system prompt（7 阶段）

P1 — 报告体验（用户感知最大的改进）

4. 新建 `RadarChart.tsx` 组件
5. 新建 `ScoreBar.tsx` 组件
6. 新建 `AhaCard.tsx` 组件
7. 新建 `reportBuilder.ts` — Next Steps + Watch Items 生成
8. 在 `claude.ts` 中添加报告生成 prompt
9. 重构 `AuditScreen.tsx` 的报告部分 — 三层布局

P2 — 对话体验（润色）

10. 在 `AuditScreen.tsx` 的 FSM 中添加 Stage 2（history_check）
11. 新建 `FrequencySlider.tsx`（Stage 4 用）
12. 添加 motivation 追问逻辑
13. 添加中途总结反馈
14. 在 prompt 规则中添加 Echo Back + 条件幽默
15. 更新 `setupQuestions.ts`（兴趣标签 + 订阅者类型）

P3 — 数据集成（依赖 Check-in 持久化完成）

16. 把 Check-in Layer 1 摘要接入 Audit context
17. 实现 CTA 行为写回 persona
18. 基于历史记录的开场白路径分叉

Appendix A: New Type Definitions（新类型定义）

```
// 添加到 types/index.ts

export type AuditStage =
  | 'service_price'
  | 'history_check'      // 新增
  | 'motivation'
  | 'expected_usage'
  | 'friction_risk'      // 内部逻辑改名为 independent_risk，类型名保留兼容
  | 'budget_fit'
  | 'overlap_decision';

export interface AuditSlots {
  // Stage 1
  service_name?: string;
  price?: Maybe<number>;
  billing_cycle?: Maybe<'monthly' | 'yearly' | 'weekly'>;
  price_monthly?: number;
```

```
  // Stage 2（新增）
  history_status?: 'first_time' | 'cancelled' | 'upgrading' | 'free_trial';
  cancel_reason?: 'price' | 'low_usage' | 'found_alternative' | 'other';
  free_trial_limiting?: boolean;
  history_source?: 'database' | 'user_reported';  // 新增：记录信息来源
  previous_duration_months?: number;                // 新增：来自数据库记录

  // Stage 3
  primary_motivation?: string;
  motivation_note?: string;
  motivation_clarity?: 'scenario_specific' | 'problem_aware' | 'category_only' |
'none';  // 新增：Claude 分类

  // Stage 4
  expected_frequency?: Maybe<0 | 1 | 2.5 | 4.5 | 7>;

  // Stage 5
  budget_feel?: Maybe<'comfortable' | 'tight' | 'no_way'>;

  // Stage 6
  overlap_level?: Maybe<'none' | 'some' | 'significant' | 'high'>;

  // Stage 7（重新设计）
  top_risk?: Maybe<'none' | 'free_enough' | 'forget_cancel' | 'interest_decay' |
'impulse'>;
  additional_risk_note?: string;

  // Final gate
  additional_notes?: string;
}

export type AuditRecommendationType =
  | 'subscribe_now'
  | 'start_trial'
  | 'watchlist'
  | 'alternative';

export interface AuditReport {
  id: string;
  user_id: string;
  service_name: string;
  cost: string;
  billing_cycle: string;
  category: string;

  // 评分
  score: number;
  score_breakdown: {
    need_fit: number;
    usage: number;
    budget_fit: number;
    overlap: number;
    risk: number;
    uncertainty_penalty: number;
  };
  recommendation_type: AuditRecommendationType;
  uncertainty_level: 'low' | 'medium' | 'high';

  // 报告内容
  verdict: string;
  why_paragraph: string;
```

```
  next_steps: Array<{ step: number; title: string; description: string }>;
  watch_items: Array<{ icon: string; text: string }>;
  factors: Array<{ title: string; description: string; type: 'risk' | 'pass' }>;

  // 雷达图数据 (0-100 per axis)
  radar_data: {
    need_clarity: number;
    expected_use: number;
    budget_comfort: number;
    uniqueness: number;
    risk_level: number;
  };

  // 上下文
  slots: AuditSlots;
  triggered_insights: string[];
  conversation_history: ChatMessage[];
  created_at: string;
}
```

Appendix B: Unresolved Questions（遗留问题）

1. **Persona Alignment 系数校准** — 0.7~1.3 的范围需要用真实用户测试，防止强匹配时过度拔高分数
2. **取消原因的采集** — 当前 app 取消订阅时没有问原因，Lifecycle 信号质量会受影响
3. **"Spontaneous" 类型用户的处理** — 订阅冲动、取消也冲动，Lifecycle 信号可能非常嘈杂，需要特殊处理
4. **Why 段落的语言质量控制** — 需要在各种边缘情况下做 prompt A/B 测试
5. **Slider 无障碍** — 确保 VoiceOver 能用自定义 Slider 组件
6. **Layer 1 更新频率** — 每次 Check-in 后立即更新还是批量处理？需确认延迟容忍度

END OF SPEC / 文档结束

下一步： 从 P0 开始实施（types → scoring → prompt），然后 P1（报告组件），然后 P2（对话润色）。